

CONEXÃO COM BD MYSQL

1. Instalação MinGW pacote Linguagem C

- **Opção 1:** Instalar MinGW-w64 (**GCC**)
- **Opção 2:** Instalar GCC via CodeBlocks com MinGW
- **Opção 3** (**muito usada**): Instalar Build Tools do Visual Studio

Instale Visual Studio Build Tools e **marque:**

- C++ build tools
- Windows SDK
- **Opção 4 Melhor opção portable: WinLibs (GCC Portable)**

1. Baixe aqui:

<https://winlibs.com/>

2. Pegue a versão **GCC + MinGW-w64 + UCRT** (zip).

3. Extraia por exemplo para:

C:\gcc\

Vai ficar algo assim:

C:\gcc\mingw64\bin\gcc.exe

Nota: Microsoft Visual Studio/MSVC é o compilador C/C++ da Microsoft para Windows. Microsoft Visual C++

2. O que você precisa fazer (**independente da opção**) **PATH**

Nota: Depois de instalar o gcc, você precisa adicionar ao **PATH**.

Como adicionar no PATH (Windows 10/11**)**

1. Aperte **Win + R**
2. Digite: **sysdm.cpl**
3. Aba **Avançado**
4. **Variáveis de Ambiente**
5. Em **Path** → Editar
6. Adicionar o caminho do gcc
7. OK em tudo

3. Teste final no **PowerShell/CMD** do Windows

- `gcc --version`

Se funcionar, **compile**: `gcc app.c -o app.exe`
`.\app.exe`

4. IDE VSCode **Versão ZIPADA**

<https://code.visualstudio.com/download>

- Descompactar
- Criar uma pasta de projetos, **wsprojetos**
- appAcademico

5. IDE VSCode – Configuração Opções

- **Opção A: rodar direto pelo caminho completo** funciona sem PATH.

No PowerShell do VSCode:

```
C:\gcc\mingw64\bin\gcc.exe app.c -o app.exe  
.\app.exe
```

- **Opção B: adicionar PATH só temporário (sem mexer no Windows)**

No PowerShell do VSCode:

```
$env:Path += ";C:\gcc\mingw64\bin"  
gcc --version  
gcc app.c -o app.exe  
.\app.exe
```

Esse **PATH** vale só enquanto o terminal estiver aberto.

▪ **Opção C: Configurar Task VSCode (recomendado); Linkedição**

No VSCode crie a pasta **,vscode** e o arq tasks.json .vscode/**tasks.json**:

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "Build C Portable MySQL",
      "type": "shell",
      "command": "C:\\gcc\\mingw64\\bin\\gcc.exe",
      "args": [
        "crud.c",
        "-o",
        "crud.exe",
        "-I",
        "${workspaceFolder}\\mysql\\include",
        "-L",
        "${workspaceFolder}\\mysql\\lib",
        "-lmysql"
      ],
      "group": "build",
      "problemMatcher": ["$gcc"]
    }
  ]
}
```

Nota: Compilar com, **Ctrl + Shift + B**

6. Rodar o Executável

Nota: Depois de **compilar+linkeditar**, execute Terminal:

.\app.exe

Nota: Senão rodar, no VSCode vá em: Settings → pesquise: code runner:

Code Runner: **desative**, Run In Terminal ou **desabilite** a extensão Code Runner.

7. Senão Conectar com o BD ou erro DLL – Analisar Compatibilidade DLL + MinGW

Nota: Seu código **compilou**, mas na hora de **linkar** o executável o GCC **não achou a biblioteca do MySQL (libmysql)**. Ou seja: No projeto incluiu o **mysql.h**, mas **não linkou** com a lib.

7.1. 1ª Tentativa **Compilar objeto + linkedição biblioteca MYSQL**

- criar a pasta **mysql** dentro do **projeto C**, copiar do
- copiar a pasta **include** do **mysql** instalado para dentro da pasta **mysql**
- criar a pasta **lib** dentro do **mysql**
- copiar a **DLL**, **libmysql.dll**, para a pasta do seu **projeto C**;
- copiar a **DLL**, **libmysql.dll**, para a pasta do seu **projeto C**, pasta **mysql\lib**;
- Gerar o DEF pelo **CMD** do Windows (**não PowerShell**)
- cd C:**wsprojetos**\appAcademico\mysql\lib
C:\gcc\mingw64\bin\pexports.exe libmysql.dll > **libmysql.def**
- C:\gcc\mingw64\bin\dlltool.exe -d libmysql.def -l **libmysql.a**

7.2. 2ª Tentativa **Compilar objeto + linkedição biblioteca MYSQL**

C:\gcc\mingw64\bin\dlltool.exe --export-all-symbols -D libmysql.dll
-l libmysql.a

7.3. 3ª Tentativa **Compilar objeto + linkedição biblioteca MYSQL**

1. Vá para a pasta da DLL

Abra PowerShell:

```
cd C:\Users\heleno\Downloads\wspython\appAcademico\mysql\lib
```

2. Gere o .def

Execute:

```
gendef libmysql.dll ou se der erro: pexports libmysql.dll > libmysql.def
```

Isso criará:

```
libmysql.def
```

3. Gere a biblioteca correta

Execute:

```
dlltool --dllname libmysql.dll --def libmysql.def --output-lib libmysql.a
```

4. Compile novamente no VSCode

CTRL + B + Build C Portable MySQL

7.4. 4ª Tentativa **Compilar objeto + linkedição biblioteca MYSQL**

Solução com outro Connector MariaDB para Server MySQL

Baixe o MariaDB Connector/C para Windows x64:

[MariaDB Connector/C Download](#)

Escolha:

- Windows
- **ZIP package**
- x64

Depois

Extraia para:

C:\mariadb

Você terá:

C:\mariadb\include
C:\mariadb\lib

e principalmente:

C:\mariadb\lib\libmariadb.dll.a

Essa biblioteca funciona **PERFEITAMENTE** com MinGW.

Troque o include do código

De:

```
#include  
<C:/Users/helena/Downloads/wspython/appAcademico/mysql/include/m  
ysql.h>
```

Para:

```
#include <mysql.h>
```

Compile

```
gcc crud.c -o crud.exe -IC:\mariadb\include -LC:\mariadb\lib -lmariadb
```

Resultado esperado

Conectado ao **MySQL** com sucesso!

Por que isso resolve?

O **MySQL oficial do Windows** normalmente fornece:

- .lib para Visual Studio/MSVC

Já o **MinGW GCC** precisa:

- .a
- .dll.a

O **MariaDB Connector/C** já vem pronto para **GCC/MinGW**.

Apenas:

- troque o **include**
- troque **-lmysql** por **-lmariadb**

O problema **NÃO** é **MySQL** em si.

- MinGW/GCC no Windows
- **incompatibilidade** de bibliotecas **.lib** do **Visual Studio**

Resumo

Visual Studio / MSVC

Compilador da Microsoft

usa: **.lib**

GCC / MinGW

Compilador GNU

usa: **.a**
.dll.a

Nota: Desenvolvimento **C/C++ no Windows usando MySQL**, o **Microsoft Visual Studio** normalmente oferece a integração mais simples e estável.

Principalmente porque:

- o **MySQL** oficial fornece bibliotecas **.lib** para **MSVC**
- não há incompatibilidade de **linker**
- configuração é mais direta
- melhor debugging no Windows
- integração nativa com **DLLs** Windows

Nota: Se aparecer isso:

Executing task: gcc ...
* Terminal will be reused...

e não aparecer erro, significa:

compilou e linkou
gerou o app.exe